<?php

UNIDAD DIDÁCTICA 02

Tutorial 01
Clase Perro

Clases, constructores, métodos y excepciones

PHP · POO · Métodos · Excepciones

UD 02 · DWES · PHP 8.1+

Tutorial 01 · Clase Perro y programación orientada a objetos

Objetivo: Vamos a construir una clase `Perro` y un punto de entrada que crea objetos, llama a sus métodos y muestra la información en el navegador. Al terminar, el resultado es una página con dos perros, un cumpleaños y un error controlado. Los archivos completos aparecen en cada paso; síguelos en orden.

Antes de comenzar

Utiliza PHP 8.1 o posterior y un servidor web local (por ejemplo, Apache de XAMPP). Crea una carpeta de proyecto dentro de `htdocs/UD2/` y **escribe los archivos con el contenido mostrado en cada paso**. El directorio `Codigo/` contiene esos mismos archivos listos para ejecutar.

La estructura de trabajo será:

```
htdocs/
└── UD2/
    └── Perro/
        ├── Perro.php
        └── index.php
```

Abre al final `http://localhost/UD2/Perro/index.php`.

Paso 1. Crear la clase `Perro.php`

Crea `Perro.php`. La palabra `class` define un nuevo tipo de objeto. En el constructor `__construct()` aparecen cuatro propiedades: nombre, edad, raza y sexo. Son `private`, por lo que los demás archivos no pueden modificarlas directamente. Los valores a la derecha de `=` son predeterminados: `new Perro()` funciona sin pasar argumentos y `new Perro('Roco', 4, 'Border Collie', 'Macho')` sustituye esos valores.

En el cuerpo del constructor se comprueba que la edad no sea negativa. Si lo es, se lanza una `InvalidArgumentException`, que indica que los datos de creación no son válidos.

Después se definen cuatro métodos: `ladrar()` devuelve una cadena, `calcularEdadHumana()` devuelve un entero, `cumplirAnios()` aumenta la edad y devuelve la nueva, y `__toString()` construye la descripción textual. Todos **devuelven resultados**, sin usar `echo` dentro de la clase. El factor siete de la edad humana es únicamente un cálculo didáctico.

Archivo `Codigo/Perro.php`

```php
<?php
declare(strict_types=1);

class Perro
{
    public function __construct(
        private string $nombre = 'Sin nombre',
        private int $edad = 0,
        private string $raza = 'Mestizo',
        private string $sexo = 'Sin especificar'
    ) {
        if ($edad < 0) {
            throw new InvalidArgumentException('La edad no puede ser negativa');
        }
    }
```

```
    public function ladrar(): string
    {
        return '¡Guau, guau!';
    }

    public function calcularEdadHumana(): int
    {
        return $this->edad * 7;
    }

    public function cumplirAnios(): int
    {
        $this->edad++;
        return $this->edad;
    }

    public function __toString(): string
    {
        return "{$this->nombre} · {$this->raza} · {$this->edad} años "
            . "(equivalencia didáctica: {$this->calcularEdadHumana()}) · {$this->sexo}";
    }
}
```

Fíjate en `$this->edad`: `$this` representa el objeto concreto sobre el que se ejecuta el método. Por eso dos objetos `Perro` tienen edades independientes. `__toString()` se utiliza cuando conviertes el objeto a texto con `(string) $perro`.

Paso 2. Crear `index.php` y mostrar los resultados

Crea `index.php` en la misma carpeta. La primera línea `require_once` carga la definición de `Perro`, y `__DIR__` hace que la ruta sea relativa al propio archivo. Luego el array `$perros` contiene dos **instancias distintas**.

El `foreach` recorre ambos objetos. Aquí sí utilizamos `echo`, porque `index.php` es el punto de entrada y su responsabilidad es mostrar resultados. `htmlspecialchars()` protege la salida HTML: convierte caracteres como `<` y `>` en texto seguro. Después, sobre el segundo perro, se llama a `cumplirAnios()`. Finalmente se provoca una edad inválida y se captura la excepción con `try/catch` para evitar una pantalla de error.

Archivo `Codigo/index.php`

```
<?php
declare(strict_types=1);
require_once __DIR__ . '/Perro.php';

$perros = [
    new Perro(),
    new Perro('Roco', 4, 'Border Collie', 'Macho')
];

foreach ($perros as $perro) {
    echo htmlspecialchars((string) $perro, ENT_QUOTES, 'UTF-8') . '<br>';
    echo htmlspecialchars($perro->ladrar(), ENT_QUOTES, 'UTF-8') . '<br>';
}

echo '<hr>';
echo 'Roco cumple años: ' . $perros[1]->cumplirAnios() . '<br>';
echo htmlspecialchars((string) $perros[1], ENT_QUOTES, 'UTF-8') . '<br>';

try {
    new Perro('Error', -2);
} catch (InvalidArgumentException $error) {
```

```
    echo htmlspecialchars($error->getMessage(), ENT_QUOTES, 'UTF-8');
}
```

No se necesita una base de datos ni un formulario. Todas las pruebas se ejecutan al abrir esta URL. El archivo de la clase no produce ninguna salida por sí solo: la salida se genera en `index.php`.

Resultado y comprobación

Abre `http://localhost/UD2/Perro/index.php`. Debes ver primero un perro con los valores predeterminados y después a Roco con 4 años; ambos devuelven «¡Guau, guau!». Después aparece «Roco cumple años: 5» y su descripción actualizada. Por último aparece el mensaje «La edad no puede ser negativa», sin detener la página con un error no controlado.

Lo importante: el constructor prepara cada objeto; sus métodos manejan el estado o devuelven información; `index.php` decide qué imprimir.